

EXPERIMENT NO. 4

FREQUENCY DOMAIN IMAGE FILTERING USING FOURIER TRANSFORM

```
In [3]: import cv2
import numpy as np
import matplotlib.pyplot as plt

# 1. LOAD GRAYSCALE IMAGE
image_path = input("Enter the path of grayscale image: ")
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if image is None:
    print("Error: Image could not be loaded!")
    exit()

print("Image loaded successfully!")
print("Image Shape:", image.shape)

# 2. DISPLAY ORIGINAL IMAGE
plt.imshow(image, cmap="gray")
plt.title("Original Grayscale Image")
plt.axis("off")
plt.show()

# 3. COMPUTE DISCRETE FOURIER TRANSFORM (DFT)
dft = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
print("DFT calculated successfully.")

# 4. SHIFT ZERO FREQUENCY TO CENTER
dft_shift = np.fft.fftshift(dft)
print("Frequency spectrum shifted to center.")

# 5. CALCULATE MAGNITUDE SPECTRUM
magnitude = cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1])
magnitude_spectrum = 20 * np.log(magnitude + 1)

# 6. DISPLAY FREQUENCY SPECTRUM
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")
plt.show()

# 7. CREATE LOW-PASS FILTER
rows, cols = image.shape
crow, ccol = rows // 2, cols // 2
radius = 50

low_pass_mask = np.zeros((rows, cols, 2), np.uint8)
cv2.circle(low_pass_mask, (ccol, crow), radius, (1, 1), -1)

print("Low-Pass Filter created.")
```

```
# 8. APPLY LOW-PASS FILTER
low_pass_dft = dft_shift * low_pass_mask

# 9. INVERSE FOURIER TRANSFORM FOR LOW-PASS
low_shift = np.fft.ifftshift(low_pass_dft)
low_image = cv2.idft(low_shift)

low_image = cv2.magnitude(
    low_image[:, :, 0],
    low_image[:, :, 1]
)

low_image = cv2.normalize(
    low_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# 10. DISPLAY LOW-PASS FILTERED IMAGE
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filtered Image")
plt.axis("off")
plt.show()

# 11. CREATE HIGH-PASS FILTER
high_pass_mask = np.ones((rows, cols, 2), np.uint8)
cv2.circle(high_pass_mask, (ccol, crow), radius, (0, 0), -1)

print("High-Pass Filter created.")

# 12. APPLY HIGH-PASS FILTER
high_pass_dft = dft_shift * high_pass_mask

# 13. INVERSE FOURIER TRANSFORM FOR HIGH-PASS
high_shift = np.fft.ifftshift(high_pass_dft)
high_image = cv2.idft(high_shift)

high_image = cv2.magnitude(
    high_image[:, :, 0],
    high_image[:, :, 1]
)

high_image = cv2.normalize(
    high_image, None, 0, 255, cv2.NORM_MINMAX
).astype(np.uint8)

# 14. DISPLAY HIGH-PASS FILTERED IMAGE
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filtered Image")
plt.axis("off")
plt.show()

# 15. DISPLAY ALL RESULTS TOGETHER
```

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(image, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(magnitude_spectrum, cmap="gray")
plt.title("Magnitude Spectrum")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(low_image, cmap="gray")
plt.title("Low-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(high_image, cmap="gray")
plt.title("High-Pass Filter")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(low_pass_mask[:, :, 0], cmap="gray")
plt.title("Low-Pass Mask")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(high_pass_mask[:, :, 0], cmap="gray")
plt.title("High-Pass Mask")
plt.axis("off")

plt.tight_layout()
plt.show()

# 16. SAVE OUTPUT IMAGES
cv2.imwrite("original_image.jpg", image)

cv2.imwrite(
    "magnitude_spectrum.jpg",
    cv2.normalize(
        magnitude_spectrum, None, 0, 255, cv2.NORM_MINMAX
    ).astype(np.uint8)
)

cv2.imwrite("low_pass_filtered.jpg", low_image)
cv2.imwrite("high_pass_filtered.jpg", high_image)
```

Image loaded successfully!

Image Shape: (1024, 1536)

Original Grayscale Image

Original Image



(Grayscale)

Low-Pass Filters (Smoothing & Noise Reduction)

Gaussian Blur



(Reduces noise, preserves structure)

Median Filter



(Removes salt and pepper noise, keeps edges)

Average Filter



(Simple smoothing, can blur edges)

High-Pass Filters (Edge Enhancement & Feature Detection)

Laplacian Filter



(Enhances edges and fine details)

Sobel X (Vertical Edges)



(Detects vertical edges)

Sobel Y (Horizontal Edges)



(Detects horizontal edges)

Edge Enhanced (Laplacian)



(Sharper details and edges)

Salt-and-Pepper Noise Removal (Median Filter)

Original



(Clean image)

Noisy Image



(Salt and pepper noise)

Median Filtered



(Noise removed, edges preserved)

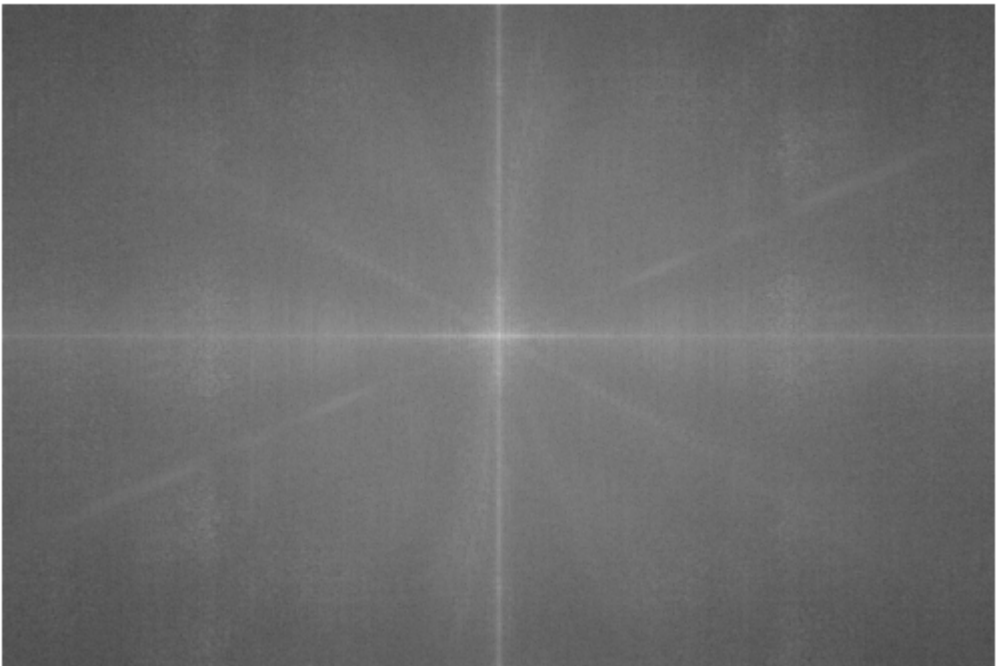
Filter Comparison					
Filter	Type	Noise Reduction	Edge Preservation	Edge Enhancement	Main Application
Gaussian	Low-pass	Good	Good	Low	Smoothing
Median	Low-pass	Excellent (Impulse noise)	Very Good	Low	Salt and pepper noise removal
Average	Low-pass	Good	Poor	Low	Simple smoothing
Laplacian	High-pass	Poor	Moderate	Excellent	Sharpening, edge enhancement
Sobel X	High-pass	Poor	Good	Excellent	Vertical edges
Sobel Y	High-pass	Poor	Good	Excellent	Horizontal edges

Key Observations

1. Gaussian filtering provides smooth results and is effective for reducing random noise.
2. Median filtering is highly effective against salt and pepper noise and preserves edges better than average filtering.
3. Average filtering smooths the image but tends to blur object boundaries.
4. Laplacian filtering emphasizes edges and fine details but can also amplify noise.
5. Sobel X detects vertical edges, while Sobel Y detects horizontal edges.
6. Low-pass filters are suitable for noise reduction and smoothing, whereas high-pass filters are useful for edge detection and sharpening.

DFT calculated successfully.
Frequency spectrum shifted to center.

Magnitude Spectrum



Low-Pass Filter created.

127.0.0.1:5500/labsheet-4.html

4/8

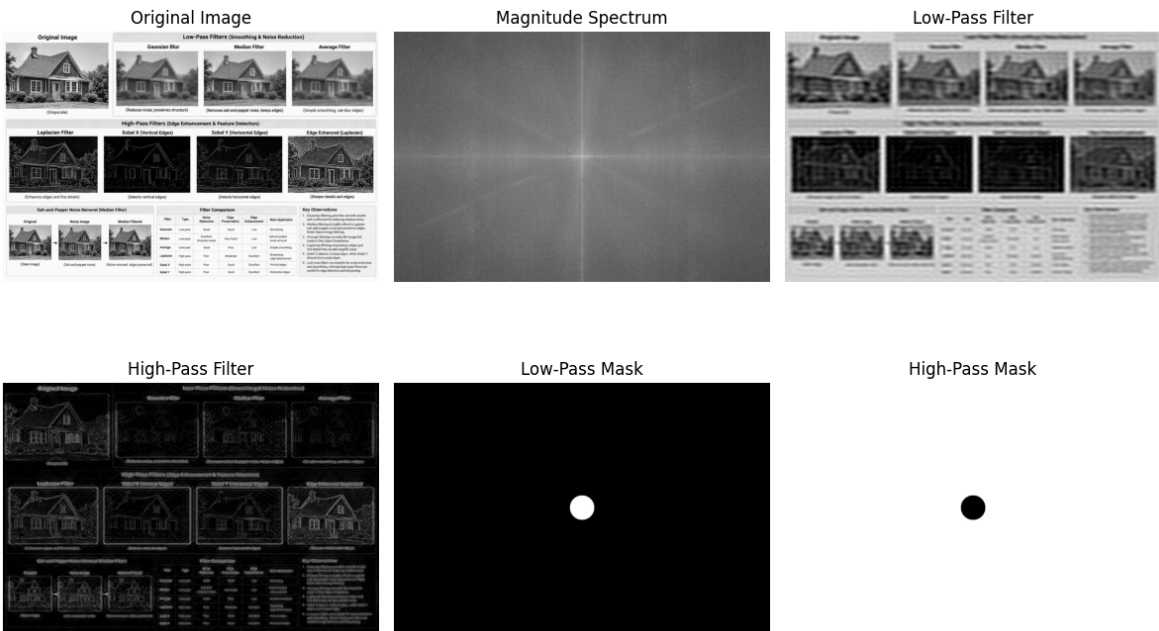
Low-Pass Filtered Image



High-Pass Filter created.

High-Pass Filtered Image





Out[3]: True

Questions and Answers

1. What is the Fourier Transform? Why is it important in digital image processing?

The Fourier Transform is a mathematical technique that converts an image from the **spatial domain** into the **frequency domain**. It represents an image using different frequency components.

It is important because it helps analyze image frequencies and perform operations such as filtering, noise removal, image enhancement, and restoration.

2. Differentiate between the spatial domain and the frequency domain.

Spatial Domain	Frequency Domain
Works directly with image pixels.	Works with frequency components.
Represents intensity at each pixel location.	Represents low and high frequency information.
Uses filters such as Average, Gaussian, and Median.	Uses Fourier Transform and frequency filters.
Suitable for local image processing.	Suitable for analyzing and modifying frequency components.

3. What is the significance of the Discrete Fourier Transform (DFT) in image processing?

The **Discrete Fourier Transform (DFT)** converts a digital image from the spatial domain into the frequency domain. It represents the image as a combination of different frequency components.

DFT is useful for image filtering, noise reduction, image restoration, compression, and frequency analysis.

4. Explain the purpose of shifting the zero-frequency component to the center of the frequency spectrum.

In the Fourier spectrum, the zero-frequency or **DC component** is normally located at the corner of the image. The `fftshift` operation moves this component to the center.

This makes the frequency spectrum easier to visualize and analyze. Low frequencies appear near the center, while high frequencies appear farther from the center.

5. Compare Low-Pass Frequency Filters and High-Pass Frequency Filters with suitable applications.

Low-Pass Frequency Filter	High-Pass Frequency Filter
Allows low-frequency components to pass.	Allows high-frequency components to pass.
Reduces high-frequency details and noise.	Enhances edges and fine details.
Produces a smoother image.	Produces a sharper image.
Used for image smoothing and noise reduction.	Used for edge enhancement and sharpening.
Example: Gaussian Low-Pass Filter.	Example: High-Pass Filter.

6. What is the role of the Inverse Fourier Transform (IDFT) in image reconstruction?

The **Inverse Fourier Transform (IDFT)** converts an image from the frequency domain back into the spatial domain.

After applying a frequency-domain filter, the modified frequency components are transformed back using IDFT to obtain the processed image. Therefore, it is essential for reconstructing the final image.

7. Why is frequency domain filtering preferred for certain image enhancement tasks?

Frequency domain filtering provides direct control over different frequency components of an image. Low frequencies can be reduced or preserved for smoothing, while high frequencies can be enhanced for sharpening and edge detection.

It is especially useful for removing periodic noise and performing advanced image enhancement operations.

8. Mention four real-world applications where Fourier Transform is used in computer vision and image analysis.

Four real-world applications are:

- 1. **Medical Imaging:** Image reconstruction and analysis in MRI and CT scans.
- 2. **Image Restoration:** Removing noise and unwanted frequency components.
- 3. **Image Enhancement:** Sharpening and smoothing images.
- 4. **Computer Vision:** Frequency-based feature analysis and pattern recognition.

9. Compare frequency domain filtering with spatial domain filtering based on computational efficiency and practical applications.

Frequency Domain Filtering	Spatial Domain Filtering
Uses Fourier Transform and inverse transform.	Works directly on pixels.
Can be efficient for large convolution operations using FFT.	Simple and efficient for small kernels.
More complex to implement.	Easier to implement.
Useful for periodic noise and frequency-based enhancement.	Useful for smoothing, sharpening, and edge detection.
Requires transformation between domains.	Directly processes image pixels.

10. How does frequency domain filtering improve the performance of image restoration and feature extraction techniques?

Frequency domain filtering allows unwanted frequency components such as noise to be removed while preserving useful image information. It can also enhance high-frequency components that contain edges and fine details.

As a result, the restored image becomes clearer and feature extraction methods can identify important edges, textures, and patterns more effectively.